National University of Computer and Emerging Sciences

OS Project

Group Members

Abid Zafar……23k-0835

Supervised by

Anum Hamid

FAST School of Computing

National University of Computer and Emerging Sciences

Karachi Pakistan

4/26

Introduction

The Custom Command Line Shell project is a simplified UNIX-like shell developed in the C programming language to replicate fundamental shell operations and demonstrate practical concepts of Operating Systems, especially process creation, command execution, synchronization, and signal handling.

This project was designed as part of the CS2004 Operating Systems coursework to provide hands-on understanding of how command-line interpreters function internally. The shell accepts user input, parses commands into executable arguments, creates child processes using fork(), executes commands through execvp(), and synchronizes parent-child execution using waitpid().

The project began in April 2026 and was completed during the Spring semester under FAST School of Computing. It was developed individually for academic purposes to fulfill mandatory project requirements while incorporating additional shell features for enhanced functionality.

The shell supports both absolute and relative path execution, enabling users to run commands like /bin/ls or ls. Additional enhancements include built-in commands (cd, pwd, exit), command history, background execution using &, signal handling for Ctrl+C, and environment variable expansion using $VARIABLE.

This report covers project features, technologies used, code implementation, and the final system evaluation.

Features

Mandatory Features

Read and Parse User Input

The shell continuously prompts the user using myShell> and reads commands via fgets(). Input is tokenized using strtok() to separate commands and arguments.

Execute Commands using fork() and execvp()

The shell creates a child process for each external command.

- fork() creates a duplicate process
- execvp() replaces the child process image with the requested command

Parent Process Waits using waitpid()

The parent shell waits for foreground processes to complete before accepting the next command.

Handle Invalid Commands Gracefully

If a command is not recognized, execvp() fails and an appropriate error message is displayed using perror().

Support Absolute and Relative Paths

Users can execute commands from:

- Absolute paths: /bin/ls
- Relative paths: ./program

Additional Features Implemented

Built-in Commands

- cd → Change directory
- pwd → Display current working directory
- exit → Terminate shell
- history → Display previously entered commands

Command History

Custom Command Line Shell

Stores up to 100 commands using a 2D character array.

Background Execution

Commands ending with & execute in the background without blocking the shell.

Signal Handling (Ctrl+C)

Intercepts SIGINT and prevents shell termination, reminding users to type exit.

Environment Variable Expansion

Supports variables like $HOME or $PATH.

Technology Used

Programming Language

- **C Language**

Libraries Used

- stdio.h → Input/output functions
- stdlib.h → General utilities
- string.h → String manipulation
- unistd.h → UNIX system calls
- sys/wait.h → Process synchronization

- signal.h → Signal handling

System Calls and Core Functions

- fork()
- execvp()
- waitpid()
- getcwd()
- chdir()
- signal()
- getenv()

Code Snippets

Signal Handling

```
void handle_sigint(int sig) {
   printf("\nUse 'exit' to quit the shell.\n");
   printf("myShell> ");
   fflush(stdout);
}
```

This function prevents abrupt shell closure when Ctrl+C is pressed.

Command Parsing

```
void parse_input(char input[], char *args[], int *background) {
   int i = 0;
   char *token;

   *background = 0;
   token = strtok(input, " ");

   while (token != NULL) {
      if (token[0] == '$') {
```

```
            char *env = getenv(token + 1);
            if (env != NULL) {
                args[i++] = env;
            }
        } else {
            args[i++] = token;
        }

        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    if (i > 0 && strcmp(args[i - 1], "&") == 0) {
        *background = 1;
        args[i - 1] = NULL;
    }
}
```

This section tokenizes input, expands environment variables, and checks for background execution.

Process Execution

```
pid_t pid = fork();

if (pid == 0) {
    execvp(args[0], args);
}
else {
    waitpid(pid, NULL, 0);
}
```

This demonstrates process creation and synchronization.

Conclusion

The Custom Command Line Shell successfully fulfills all mandatory project requirements and implements five additional advanced features, making it a robust educational shell. Through this project, important Operating System concepts such as process creation, synchronization, command parsing, environment management, and signal handling were practically implemented.

The shell behaves similarly to standard UNIX shells for basic operations and provides an effective foundation for understanding shell internals. While advanced functionalities such as pipes and I/O redirection were not implemented, the project establishes a strong base for future upgrades.

Overall, the project was completed successfully and on schedule, demonstrating practical mastery of UNIX process management and shell design principles.